PROGINEER · DAILY TASK 2 · REPORT

Dockerizing the Donation Management API

Izzaldeen · October 2026 · ASP.NET Core (.NET 10) · EF Core 10 · SQL Server 2022 · Docker Engine + Compose on Ubuntu (WSL2)

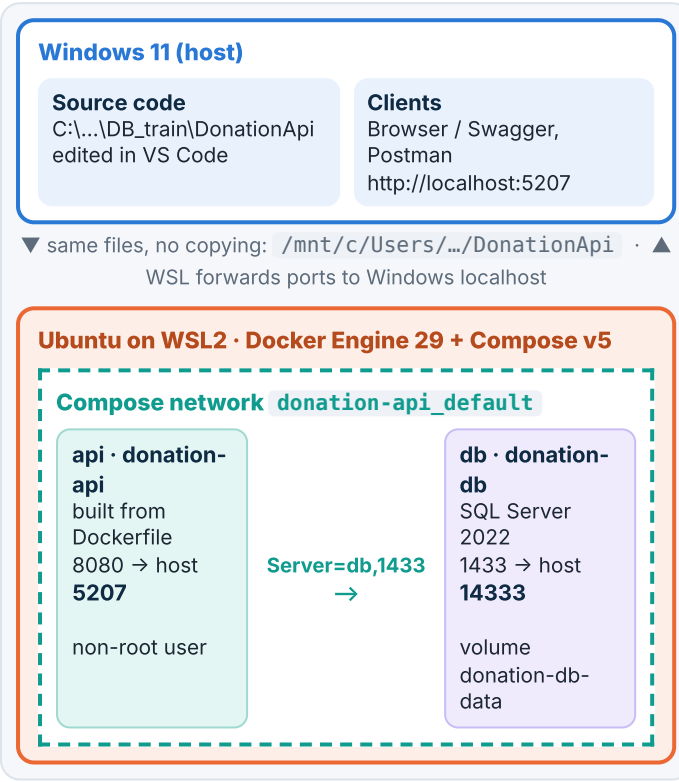
2
containers: **api** + **db**, one Compose network

1 command
docker compose up -d --build

10 / 10
API checks passed against the containers

0
credentials in source code (all from `.env`)

1 Where Docker runs and how it is connected



Docker runs inside Ubuntu (WSL2), not as a Windows program. The project stays in its normal Windows folder; Ubuntu sees it at `/mnt/c/…`, so Compose is started from that folder and builds directly from the Windows files.

Inside Docker, Compose creates a private network. Containers find each other **by service name**, so the API reaches SQL Server at `Server=db,1433` (not `localhost`).

Published ports are forwarded by WSL to Windows, so Swagger and Postman keep using **http://localhost:5207**, the same URL as before Docker.

Container	Role	host : container
donation-api	ASP.NET Core API	5207 : 8080
donation-db	SQL Server 2022	14333 : 1433

2 What was added to the project

File	Purpose
`Dockerfile`	Multi-stage build: `dotnet/sdk:10.0` restores and publishes; the final `dotnet/aspnet:10.0` image holds only the published app, runs as a non-root user on port 8080.
`compose.yaml`	Services `db` and `api`. `db` has a health check (`sqlcmd SELECT 1`); `api` uses `depends_on: condition: service_healthy`, so it starts only when SQL Server accepts connections. Named volume keeps the data.
`.dockerignore`	Keeps `bin/`, `obj/`, `.git`, `.env`, Postman and docs out of the build context.
`.env.example` / `.env`	Database password, name and ports. `.env` is git-ignored; the example file is committed as a template.
`Program.cs`	One small addition: when `Database__MigrateOnStartup=true` (set only in Compose) the API applies the EF Core migrations on startup, retrying while SQL Server starts. API logic and schema unchanged.
`README.md`	New Docker section: create `.env`, build/start, check, stop.

3 Configuration without hardcoded credentials

```
# .env (git-ignored)
DB_PASSWORD=********
DB_NAME=DonationDb
API_PORT=5207
```

```
# compose.yaml → api.environment ("__" = ":", overrides appsettings.json)
ConnectionStrings__Default: "Server=db,1433;Database=${DB_NAME};
  User Id=sa;Password=${DB_PASSWORD};TrustServerCertificate=True"
Database__MigrateOnStartup: "true"
```

4 How to build, start and stop (from Ubuntu)

```
cd "/mnt/c/Users/Asus i7/Desktop/DB_train/DonationApi"
cp .env.example .env              # first time only, then set DB_PASSWORD
docker compose up -d --build      # build the API image, start db → (healthy) → api
docker compose ps                 # both containers Up, db (healthy)
docker compose logs -f api        # "Database migrations applied." → "Now listening on: http://[::]:8080"
docker compose down               # stop and remove containers + network (data volume kept)
```

5 Results

```
izz@DESKTOP-EE3SHD7:/mnt/c/Users/Asus i7/Desktop/DB_train/DonationApi$ docker compose up -d --build
[+] Building 1.3s (17/17) FINISHED
 => [internal] load local bake definitions                                    0.0s
 => => reading from stdin 551B                                                0.0s
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 760B                                          0.0s
 => [internal] load metadata for mcr.microsoft.com/dotnet/sdk:10.0            0.4s
 => [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:10.0         0.4s
 => [internal] load .dockerignore                                             0.0s
 ... (cached build steps, export, naming to docker.io/library/donation-api:latest) ...
 => resolving provenance for metadata file                                    0.0s
[+] up 3/3
 ✓ Image donation-api:latest Built                                            1.5s
 ✓ Container donation-db     Healthy                                          0.5s
```

docker compose up -d --build: both build stages (sdk → aspnet), image `donation-api:latest` built, `donation-db` reports **Healthy** before the API container starts.

```
izz@DESKTOP-EE3SHD7:/mnt/c/Users/Asus i7/Desktop/DB_train/DonationApi$ docker compose ps
NAME           IMAGE                                                  COMMAND                  SERVICE   CREATED
       STATUS                    PORTS
donation-api   sha256:8c2e39dbffc076200c82c182307c65683a868fcf75ff234bba660140d9e6ebe6   "dotnet DonationApi.…"   api       8 minute
s ago   Up 8 minutes              0.0.0.0:5207->8080/tcp, [::]:5207->8080/tcp
donation-db    mcr.microsoft.com/mssql/server:2022-latest             "/opt/mssql/bin/laun…"   db        8 minute
s ago   Up 8 minutes (healthy)   0.0.0.0:14333->1433/tcp, [::]:14333->1433/tcp
izz@DESKTOP-EE3SHD7:/mnt/c/Users/Asus i7/Desktop/DB_train/DonationApi$
```

docker compose ps: `donation-api` Up on `0.0.0.0:5207→8080` ; `donation-db` Up **(healthy)** on `0.0.0.0:14333→1433` .

```
izz@DESKTOP-EE3SHD7:/mnt/c/Users/Asus i7/Desktop/DB_train/DonationApi$ docker compose down
[+] down 3/3
 ✓ Container donation-api       Removed
 ✓ Container donation-db        Removed
 ✓ Network donation-api_default Removed
izz@DESKTOP-EE3SHD7:/mnt/c/Users/Asus i7/Desktop/DB_train/DonationApi$
```

docker compose down: both containers and the network removed cleanly.

```
$ docker compose logs api
info: DonationApi[0]
      Database migrations applied.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://[::]:8080
      Hosting environment: Development

GET http://localhost:5207/api/health
{"status":"Healthy","database":"Connected"}
```

Verification (run from Windows against the containers)	Result
API ↔ database over the Compose network (`/api/health`)	**200 · Healthy · Connected**
Schema created automatically in the empty SQL Server container	**InitialCreate + KeysetPaginationIndex applied, index present**
CRUD: create (201), read (anonymous name masked), update (200), delete	**pass**
Errors: invalid body → 400, unknown id → 404	**pass**
Pagination: page 1 + totalCount, page 2 with cursor (no overlap), bad cursor → 400	**pass**
Data persistence: `docker compose down` → `up`	**donation still present (named volume)**
Swagger UI at `http://localhost:5207/swagger`	**200**
Host dependencies needed to run	**none besides Docker (no .NET SDK, no local SQL Server)**

Repository: github.com/aldinizz2004-png/DonationApi-CRUD · committed files: Dockerfile, compose.yaml, .dockerignore, .env.example, Program.cs, README.md (the real .env is git-ignored).